Объектуудын хувьд хэрхэн бодох вэ

"Объект хандалтат үзэл баримтлалын оршил" 1-р бүлэгт та объект хандалтат (ОО) үндсэн ойлголтуудыг сурсан. Номын үлдсэн хэсэг нь эдгээр ойлголтуудыг илүү гүнзгийрүүлэн судлахаас гадна өөр хэд хэдэн ойлголтыг танилцуулсан болно. ОО дизайн эсэхээс үл хамааран сайн загварт олон хүчин зүйл нөлөөлдөг. ОО дизайны үндсэн нэгж нь Класс юм. ОО дизайны хүссэн эцсийн үр дүн нь бат бөх, ажиллагаатай объектын загвар буюу өөрөөр хэлбэл бүрэн систем юм.

Амьдралын ихэнх зүйлсийн нэгэн адил асуудалд хандах зөв эсвэл буруу дүрэм гэж байдаггүй. Ижил асуудлыг шийдвэрлэх олон янзын дүрэм байдаг. Тиймээс, ОО шийдлийг зохион бүтээх гэж оролдохдоо анх удаа төгс дизайн хийх гэж оролдоод бүү гацаарай (үргэлж сайжруулах орон зай болон өөр өөр чиглэлүүд байдаг. Асуудлыг шийдэх гэж оролдохдоо аливаа стандарт, конвенцид нийцүүлэхийг бүү оролд, учир нь бүх санаа нь бүтээлч байх явдал юм.

Үнэн хэрэгтээ, үйл явцын эхэнд тодорхой програмчлалын хэлийг бүү бодоорой. Бизнесийн хамгийн эхний даалгавар бол бизнесийн асуудлуудыг тодорхойлж, шийдвэрлэх явдал юм.

Хамгийн түрүүнд ерөнхий санаа, системийн шинжилгээ, дизайн. Бизнесийн асуудлын үндсийг ойлгосны дараа л тодорхой технологийн талаар бод. Жишээлбэл, утасгүй сүлжээг утасгүй технологигүйгээр зохион бүтээх боломжгүй. Гэсэн хэдий ч,заавал нэг технологиор шийдэх нэгээс олон програм хангамжийн шийдлийг анхаарч үзэх хэрэгтэй.

Тиймээс систем, тэр ч байтугай классуудын зохиомжийг бүтээхээсээ өмнө асуудлыг сайтар бодож, анхааралтай байх хэрэгтэй.

Энэ бүлэгт бид ОО байдлаар хэрхэн сэтгэх талаар судлах болно.

Сэтгэлгээний аливаа үндсэн өөрчлөлт нь өчүүхэн зүйл биш юм. Үүний нэг жишээ болгон, бүтэц зохион байгуулалтаас ОО хөгжил рүү шилжих талаар маш их дурдсан байдаг. Үүний нэг гаж нөлөө

Бүтэцлэгдсэн програмчлал ба объект хандалтат програмчлал нь бие биенээ үгүйсгэдэг гэсэн буруу ойлголт юм. Энэ бол тийм биш юм. Бүтэцлэгдсэн програм хангамж болон объект хандалтат ойлголт нь зэрэгцэн оршдог. Бид үнэндээ ОО програм бичихдээ бүтэцлэгдсэн бүтцийг хаа сайгүй ашиглаж байна. ОО-гаар сэтгэнэ гэдэг нь шинэ програмчлалын хэл сурах асуудал биш энд хэзээ ч if, else, while гэх мэтийн талаар буюу бичиглэлийн талаар яригдахгүй. ОО код гэж үнэн хэрэгтээ байхгүй боловч ОО дизайн руу шилжих нь сэтгэлгээний болон ерөнхий санааны дүрслэлийг шаарддаг.

FORTRAN-аас COBOL, тэр ч байтугай C руу шилжихийн тулд бид шинэ хэл сурах шаардлагатай боловч COBOL-ээс C++, C# .NET, Visual Basic .NET, эсвэл Java руу шилжихийн тулд шинэ сэтгэхүйн үйл явцыг сурахыг шаарддаг.

ОО парадигм гэдэг үг хэллэг нь толгойгоо их ажилуулахыг шаарддаг. ОО хэл рүү шилжихдээ ОО-ийн ойлголтуудыг маш сайн эзэмшсэн байх хэрэгтэй. ОО-гийн зарчим болон ойлголтуудыг маш сайн мэдэж байж түүн дээр сэтгэх боломжтой. Хэрвээ тухайн хийх системийн хувьд тодорхой парадигмын өөрчлөлт гарахгүй бол заавал ОО руу оруулах шаардлагагүй. Систем үнэхээр ОО шинж чанартай биш байх болно. Энэ тохиолдолд жишээ нь ОО ашиглахгүйгээр C++ ашиглаж болно. ОО ашиглахгүй шинж чанартай систем дээр ашиглавал систем нь объектын чиг баримжааг алдагдуулсан бүрэн эмх замбараагүй болох магадлалтай. ОО байдлаар сэтгэх мэдрэмжийг хөгжүүлэхийн тулд дараах 3 чухал зүйлийг анхаарах хэрэгтэй. Үүнд:

1.Интерфэйс (Interface) ба хэрэгжүүлэлтийн (Implementation) ялгааг мэдэх

2. Илүү хийсвэр сэтгэх (Thinking more abstractly)

3. Хэрэглэгчдэд боломжит хамгийн бага интерфэйсийг өгөх

(Giving the user the minimal interface possible)

Бид 1-р бүлэгт эдгээр ойлголтуудын заримыг авч үзсэн бөгөөд одоо илүү дэлгэрэнгүй авч үзэх болно.

1.Интерфэйс (Interface) ба хэрэгжүүлэлтийн (Implementation) ялгааг мэдэх

1-р бүлэгт бидний харсанчлан, хүчирхэг ОО загварыг бий болгох түлхүүрүүдийн нэг нь интерфейс ба хэрэгжүүлэлтийн хоорондын ялгааг ойлгох явдал юм. Иймээс класс зохион бүтээхдээ хэрэглэгч юу мэдэх ёстой, юуг мэдэх шаардлагагүй гэдэг нь амин чухал ач холбогдолтой.

Капсуляцитай холбоотой өгөгдөл нуух механизм нь чухал бус өгөгдлийг хэрэглэгчээс нуух хэрэгсэл юм.

Анхааруулга

Интерфейсийн тухай ойлголтыг график хэрэглэгчийн интерфэйс (GUI) гэх мэт нэр томъёотой андуурч болохгүй. Хэдийгээр GUI нь нэрнээс нь харахад интерфейс боловч энд ашигласан интерфейс гэсэн нэр томъёо нь илүү ерөнхий шинж чанартай бөгөөд график интерфейсээр хязгаарлагдахгүй.

1-р бүлэгт байгаа талх шарагчийн жишээг санаж байна уу? Шарсан шарагч эсвэл ямар нэгэн хэрэгсэл нь цахилгааны залгуур болох интерфэйс рүү залгагддаг. Зураг 2.1-ийг үзнэ үү.

Бүх цахилгаан хэрэгсэл нь зөв интерфэйс буюу цахилгааны залгуурыг дагаж мөрдөх замаар шаардлагатай цахилгааныг олж авдаг.

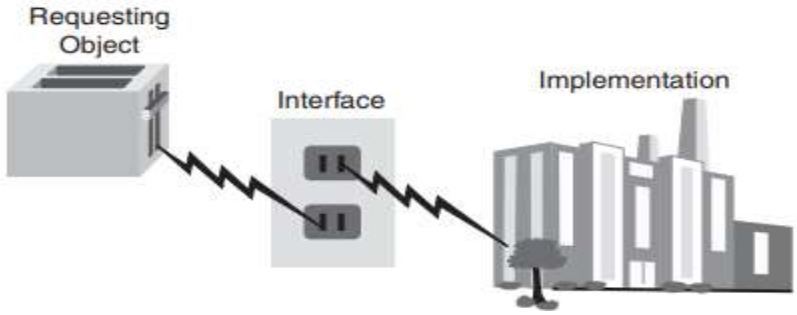
Талх шарагч нь хэрэгжүүлэлт буюу цахилгаан эрчим хүчийг хэрхэн үйлдвэрлэдэг талаар юу ч мэдэх шаардлагагүй.

Нүүрсний үйлдвэр эсвэл цөмийн үйлдвэр гээд бүх талх шарагчдад зориулсан

цахилгаан эрчим хүч үйлдвэрлэж болно—интерфэйс нь зөв, аюулгүй ажиллаж л байвал төхөөрөмж нь алийг нь ч хамаагүй.

Өөр нэг жишээ болгон автомашиныг авч үзье. Хүн болон машин хоорондын интерфейс нь жолооны хүрд, хийн дөрөө, тоормос, гал асаах унтраалга зэрэг бүрэлдэхүүн хэсгүүдийг агуулдаг.

Ихэнх хүмүүсийн хувьд гоо зүйн асуудлаас гадна машин жолоодох үед хамгийн их санаа зовдог зүйл бол машин асаах, хурдасгах, зогсох, жолоодох гэх мэт. Хэрэгжүүлэлт нь үндсэндээ жолоочийн хийх зүйл юм.



Зураг 2.1 Цахилгаан станцыг дахин сайжруулсан нь

Хэрэгжүүлэлт, үндсэндээ эолоочийн мэдэхгүй зүйл нь бөгөөд үүнд жолооч санаа зовдоггүй.

Үнэн хэрэгтээ ихэнх хүмүүс катализатор, жийргэвч гэх мэт зарим бүрэлдэхүүн хэсгүүдийг яг ямар, яаж ажилладаг гэдгийг мэдэх хэрэггүй байдаг. Гэсэн хэдий ч, энэ нь нийтлэг интерфейс учраас ямар ч жолооч жолооны хүрдийг хэрхэн ашиглахаа мэддэг, мэддэг.

Машиндаа стандарт жолооны хүрд суурилуулснаар зорилтот зах зээлд байгаа хүмүүс уг системийг ашиглах боломжтой болно гэдэгт үйлдвэрлэгчид итгэлтэй байна.

Гэсэн хэдий ч үйлдвэрлэгч жолооны хүрдний оронд джойстик суурилуулахаар шийдсэн бол ихэнх жолооч нар үүнд дургүйцэх бөгөөд машин нь чиг хандлагыг үгүйсгэх дуртай зарим хүмүүс үүнийг дэмжихгүй байх магадлалтай. Нөгөөтэйгүүр, гүйцэтгэл, гоо зүй нь өөрчлөгдөөгүй л бол дундаж жолооч үйлдвэрлэгч автомашины хөдөлгүүрийг (хэрэгжүүлэх хэсэг) өөрчилсөн эсэхийг анзаарахгүй байх болно.

Интерфэйсийн хувьд сольж болох хөдөлгүүрүүд нь бүх талаараа ижил байх ёстой гэдгийг тэмдэглэх нь зүйтэй. Дөрвөн цилиндртэй хөдөлгүүрийг найман цилиндртэй хөдөлгүүрээр солихын тулд дүрмийг өөрчлөх бөгөөд энэ тохиолодолд хөдөлгүүртэй холбогдох бусад бүрэлдэхүүн хэсгүүдтэй зохицон ажиллахгүй байх магадлалтай. Үүнтэй адилаар гүйдлийг хувьсах гүйдлээс тогтмол гүйдэл болгон өөрчлөх нь цахилгаан станцын жишээн дэх дүрэмд нөлөөлнө.

Хөдөлгүүр нь хэрэгжүүлэлтийн нэг хэсэг бөгөөд жолооны хүрд нь интерфейсийн нэг хэсэг юм.

Хэрэгжүүлэлтийн өөрчлөлт нь жолоодлонод ямар ч нөлөө үзүүлэхгүй, харин интерфейсийн өөрчлөлт нь нөлөөлнө. Үүнд жолооч нь жолоодлогын хувьд өөрчлөлтүүдийг анзаарах болно.

Жолоочийн анзаарсан энэхүү хөдөлгүүрийн өөрчлөлт нь дүрмийг зөрчиж байгааг анхаарч засч залруулах нь зүйтэй.

Жишээлбэл, эрчим хүчний мэдэгдэхүйц алдагдалд хүргэх өөрчлөлт нь интерфейсийг өөрчилж байна.

Хэрэглэгчид юу хардаг вэ?

Интерфейсүүд нь классудтай шууд холбоотой. Эцсийн хэрэглэгчид ихэвчлэн ямар ч класс хардаггүй. Хэрэглэгчид GUI эсвэл командын мөрийг хардаг. Энэ бүлэгт классын зохиосжийг ярьж байгаа учраас програмистууд классын интерфейсийг харах болно. Класс дахин ашиглах нь хэн нэгэн аль хэдийн класс бичсэн гэсэн үг юм. Тиймээс класс ашигладаг програмист хүн классаа хэрхэн зөв ажиллуулахаа мэддэг байх ёстой.

Энэхүү программист олон классудыг нэгтгэн систем үүсгэх болно. Програмист бол тухайн классын интерфейсийг ойлгох үүрэгтэй хүн юм.

Тиймээс бидний энэ хичээл дээрх хэрэглэгчид нь үндсэндээ эцсийн хэрэглэгчид биш дизайнерууд болон хөгжүүлэгчид юм. Тиймээс бид энэ хүрээнд интерфэйсийн тухай ярихдаа GUI биш харин классын интерфейсийн тухай ярих болно.

Зөв зохион байгуулалтай классуудыг интерфэйс ба хэрэгжүүлэлт гэсэн хоёр хэсэгт хуваадаг.

Интерфейс-Interface

Эцсийн хэрэглэгчдэд үзүүлэх үйлчилгээ нь интерфейсээс бүрддэг. Энэ нь зөвхөн эцсийн хэрэглэгчдэд хэрэгтэй үйлчилгээг үзүүлнэ. Хэрэглэгчдэд ямар үйлчилгээ хэрэгтэй байж болох

Хэрэв та 10 хүнийг нэг өрөөнд оруулаад тус бүрээс бие даасан дизайн хийхийг хүсэх юм бол 10 огт өөр загвар хүлээж авах боломжтой бөгөөд үүнд ямар нэгэн буруу байхгүй боловч ерөнхий дүрмээр бол классууд болон интерфейс агуулагдах ёстой.

Шарсан талхны жишээнд хэрэглэгч зөвхөн үүнийг мэдэх хэрэгтэй. Үүнд талх шарагчийг интерфэйс рүү залгасан байх ёстой (энэ тохиолдолд цахилгааны залгуур байдаг) мөн талх шарагчийг хэрхэн ажиллуулах заавар байдаг.

Хэрэглэгчийг тодорхойлох

Классуудыг үүсгэхдээ анхаарах хамгийн чухал зүйл бол тухайн классын хэрэглэгчдийг тодорхойлох явдал юм.

Хэрэгжүүлэлт-Implementation

Хэрэгжүүлэлт нь дэлгэрэнгүй мэдээллийг хэрэглэгчээс нуусан байдаг.

Хэрэгжүүлэлттэй холбоотой нэг чухал зүйл нь хэрэгжүүлэлтийг өөрчлөхөд хэрэглэгчийн кодыг өөрчлөх шаардлагагүй.

Энэ нь жаахан ойлгомжгүй мэт санагдаж болох ч энэ зорилго нь дизайн буюу зохиомжийн асуудлын гол цөм юм. Хэрэв интерфэйс зөв хийгдсэн бол хэрэгжүүлэлтийг өөрчлөхөд хэрэглэгчийн кодыг өөрчлөх шаардлагагүй.

Интерфэйс нь дүрмийг дуудаж, утгыг буцаах синтакс агуулдаг гэдгийг бид мэднэ.

Хэрэв энэ интерфэйс өөрчлөгдөөгүй бол хэрэглэгч хэрэгжүүлэлт өөрчлөгдсөн эсэх нь хамаагүй. Программист ижил синтаксийг ашиглаж, ижил утгыг олж авах боломжтой л бол энэ л чухал.

Үүнийг бид гар утас хэрэглэж байхдаа байнга хардаг.

Дуудлага хийхийн тулд интерфэйс нь энгийн бөгөөд бид дугаар руу залгадаг. Гэсэн хэдий ч үйлчилгээ үзүүлэгч төхөөрөмжөө өөрчилсөн ч хэрэглэгчийн дуудлага хийх дүрмийг өөрчлөхгүй.

Хэрэгжүүлэлт хэрхэн өөрчлөгдөхөөс үл хамааран интерфэйс нь ижил хэвээр байна.

Жнь доор үйлчилгээ үзүүлэгч интерфэйсийг өөрчилсөн нэг жишээг авч үзье.

- Хэрэглэгчийн бүсийн код өөрчлөгдсөн гэж үзье. Бүс, нутгийн кодын өөрчлөлт гэх мэт интерфейсийн үндсэн өөрчлөлтүүд нь хэрэглэгчдээс зан төлвийг өөрчлөхийг шаарддаг.

Бизнесүүд энэ төрлийн өөрчлөлтийг аль болох бага байлгахыг хичээдэг, учир нь зарим үйлчлүүлэгчид өөрчлөлтөд дургүй байх эсвэл магадгүй байнга өөрчлөгддөг, ашиглахад бэрхшээлтэй байдлыг тэвчихгүй байх тохиолдол байж болно.

Шарсан талхны жишээн дээр интерфэйс нь үргэлж цахилгаан залгуур байдаг хэдий ч хэрэгжүүлэлт нь талх шарагчинд нөлөөлөхгүйгээр нүүрсний цахилгаан станцаас атомын цахилгаан станц болж өөрчлөгдөж болох боломжтой.

Энэ жишээн дээр маш чухал нэг зүйлийг анхаарах хэрэгтэй.: Нүүрс эсвэл цөмийн станц нь интерфейсийн техникийн үзүүлэлтэд нийцсэн байх ёстой.

Нүүрсний үйлдвэр хувьсах гүйдлийн эрчим хүч үйлдвэрлэдэг бол атомын станц тогтмол гүйдлийн хүчийг үйлдвэрлэдэг бол асуудал үүсдэг.

Хамгийн гол нь хэрэглэгч болон хэрэгжүүлэлт хоёулаа интерфэйсийн тодорхойлолтод нийцсэн байх ёстой.

Интерфэйс/хэрэгжүүлэлтийн жишээ

Энгийн мэдээллийн сан уншигчийн класс үүсгэцгээе. Бид өгөгдлийн сангаас бүртгэл авах Java код бичих шаардлага гарсан гэж үзье. Бидний ярилцсанчлан, ямар ч төрлийн зохиомж хийхэд эцсийн хэрэглэгчээ мэдэх нь хамгийн чухал асуудал байдаг. Зохиомжийг хийх программист нөхцөл байдалд дүн шинжилгээ хийж, эцсийн хэрэглэгчидтэй ярилцлага хийж, төсөлд тавигдах шаардлагуудыг жагсаах хэрэгтэй. Өгөгдлийн сангийн классыг ашиглахыг хүсэж байгаа уншигчдын шаардлагууд нь дараах байдалтай байж болно. Үүнд

- Мэдээллийн сантай холболт нээх чадвартай байх ёстой.

- Мэдээллийн сантай холболтыг хааж чаддаг байх ёстой.

- Мэдээллийн баазын эхний бичлэг дээр курсорыг байрлуулах чадвартай байх ёстой.

- М мэдээллийн баазын хамгийн сүүлийн бичлэг дээр курсорыг байрлуулах чадвартай байх ёстой.

- Мэдээллийн санд байгаа бичлэгийн тоог олох чадвартай байх ёстой.

- Мэдээллийн санд илүү олон бичлэг байгаа эсэхийг тодорхойлох боломжтой байх ёстой (өөрөөр хэлбэл бид төгсгөлд байгаа бол).

- Түлхүүрийг нийлүүлснээр курсорыг тодорхой бичлэг дээр байрлуулах боломжтой байх ёстой.

- Түлхүүр нийлүүлснээр бид бичлэгийг олж авах боломжтой байх ёстой.

- Курсорын байрлал дээр үндэслэн дараагийн бичлэгийг авах боломжтой байх ёстой.

Эдгээр шаардлагуудыг харгалзан эдгээр эцсийн хэрэглэгчдэд боломжит интерфэйсийг бий болгосноор өгөгдлийн сангийн уншигчийн классыг зохион бүтээх анхны оролдлогыг хийж чадна.

Энэ тохиолдолд мэдээллийн сан уншигчийн класс нь мэдээллийн сан ашиглах шаардлагатай програмистуудад зориулагдсан болно.

Тиймээс интерфэйс нь үндсэндээ программист ашиглах программчлалын интерфейс (API) юм.

Эдгээр дүрмүүд нь үнэндээ өгөгдлийн сангийн системээр хангагдсан функцийг багтаасан wrap up-буюу ороомог юм.

Яагаад үүнийг хийх ёстой гэж? Энэ асуултыг бүлгийн дараа илүү нарийвчлан судлах болно; Үүнийг хийхийн тулд мэдээллийн баазын зарим функцийг өөрчлөх шаардлагатай юм.

Жишээлбэл, эхний ээлжинд объектуудыг хайж олон, боловсруулах шаардлагатай юм.

Мөн эдгээр обьектуудын хоорондын харилцааг мэдээллийн санд бичиж болно.

Энэхүү завсрын програмыг бичих нь дизайн, кодчилолын хувьд тийм ч энгийн зүйл биш боловч wrap up – ороомог буюу боодлын функциональ байдлын бодит жишээ юм.

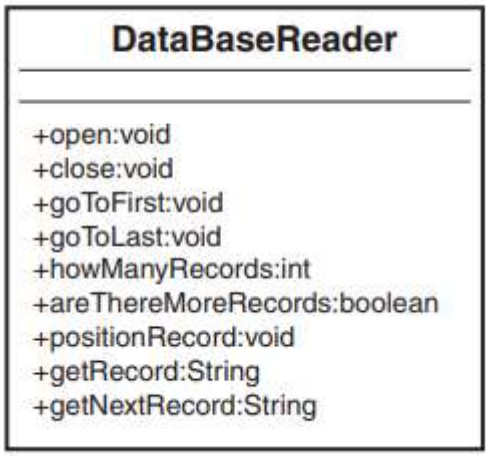
Хамгийн чухал нь кодыг өөрчлөхгүйгээр мэдээллийн сангийн удирдлагыг өөрөө өөрчлөх хүсэлтэй болж магадгүй юм.

Зураг 2.2-т DataBaseReader классын боломжит интерфэйсийг төлөөлөх классын диаграммыг үзүүлэв.

Энэ классын дүрмүүд бүгд public гэдгийг анхаарна уу (public интерфейс болох дүрмүүдын нэрний хажууд нэмэх тэмдэгтэй).

Мөн зөвхөн интерфэйсийг төлөөлдөг гэдгийг анхаарна уу; хэрэгжүүлэлт харагдахгүй байна. Энэ классын диаграм нь төслийн өмнө дурдсан шаардлагыг ерөнхийд нь хангаж байгаа эсэхийг тодорхойлоход нэг минут зарцуулж шалгаж үзээрэй.

Хэрэв диаграмм нь бүх шаардлагыг хангахгүй байгааг сүүлд нь олж мэдвэл санаа зовох хэрэггүй бөгөөд ОО дизайн нь давтагдах процесс тул үүнийг эхний удаад яг тааруулж авах шаардлагагүй.



Зураг 2.2 Нэгдсэн загварчлалын хэлний (UML) DataBaseReader класс диаграм.

Public интерфейс
Хэрэв дүрэм нь public бол хэрэглээний программист түүнд хандах боломжтой бөгөөд ингэснээр энэ нь классын интерфейсийн нэг хэсэг гэж тооцогддог.

Интерфейс гэсэн нэр томъёог түлхүүр үгтэй андуурч болохгүй

Java болон .NET-д хэрэглэгддэг интерфэйс—энэ нэр томъёог дараа авч үзэх болно

Бидний жагсаасан шаардлага бүрийн хувьд хүссэн функцийг хангах тохирох дүрэм хэрэгтэй. Үүнийг тодруулахын тулд дараах асуултуудад хариулах хэрэгтэй:

- Энэ классыг үр дүнтэй ашиглахын тулд програмист хүний хувьд энэ талаар өөр зүйл мэдэх шаардлагатай юу?

- Өгөгдлийн сангийн дотоод код нь мэдээллийн санг хэрхэн нээж байгааг мэдэх шаардлагатай юу?

- Өгөгдлийн сангийн дотоод код нь тодорхой бичлэг дээр биет байдлаар хэрхэн байрлаж байгааг мэдэх шаардлагатай юу?

- Өгөгдлийн сангийн дотоод код өөр бүртгэл үлдсэн эсэхийг хэрхэн тодорхойлдогийг мэдэх шаардлагатай юу?

Ямар ч тохиолдолд хариулт нь үнэхээр үгүй! Та эдгээр мэдээллийн алийг нь ч мэдэх шаардлагагүй.

Таны анхаарах зүйл бол та зохих өгөөжийн утгыг авч, үйлдлүүд зөв хийгдсэн байх явдал юм.

Үнэн хэрэгтээ, хэрэглээний программист нь хэрэгжүүлэлтээс дор хаяж нэг хийсвэр түвшний хол байх магадлалтай.

Програм нь мэдээллийн санг нээхийн тулд таны классудыг ашиглах бөгөөд энэ нь эргээд зохих мэдээллийн баазын API-г ажиллуулах болно.

Хамгийн бага интерфэйс
Хэдийгээр хэт туйлширсан ч гэсэн минималист интерфейсийг тодорхойлох нэг дүрэм бол эхлээд хэрэглэгчдэд public интерфэйс өгөхгүй байх явдал юм.

Мэдээжийн хэрэг, класс нь ашиггүй болно; Гэсэн хэдий ч энэ нь хэрэглэгчийг тан руу буцаж ирээд "Хөөе, надад энэ функц хэрэгтэй байна" гэж хэлэхийг албаддаг.

Дараа нь та тохиролцож болно. Тиймээс та интерфэйсийг зөвхөн хүссэн үед нь нэмнэ.

Хэрэглэгчид ямар нэгэн зүйл хэрэгтэй гэж хэзээ ч бүү бод.

Боодол үүсгэх нь хэтэрхий төвөгтэй мэт санагдаж болох ч бичих нь олон давуу талтай.

Тодруулбал, өнөөдөр зах зээл дээр олон дунд програм хангамжийн бүтээгдэхүүн байдаг.

Объектуудыг харилцааны мэдээллийн санд буулгах асуудлыг авч үзье.

Өнөөдөр зах зээл дээр ОО програмуудад тохиромжтой ОО мэдээллийн сангууд байдаг. Гэсэн хэдий ч, нэг жижиг асуудал бий: Ихэнх компаниуд хуучин харилцааны мэдээллийн санд олон жилийн өгөгдөлтэй байдаг.

Компани хэрхэн ОО технологиудыг хүлээн авч, харилцааны мэдээллийн санд мэдээллээ хадгалахын зэрэгцээ хамгийн сүүлийн үеийн дэвшилтэд байж чадах вэ?

Нэгдүгээрт, та өөрийн бүх хуучин, харилцааны өгөгдлөө цоо шинэ ОО мэдээллийн сан болгон хөрвүүлэх боломжтой.

Гэсэн хэдий ч аливаа өгөгдөл хувиргах үед цочмог (болон архаг) өвдөлтийг амссан хэн бүхэн үүнийг ямар ч үнээр хамаагүй зайлсхийх хэрэгтэй гэдгийг мэддэг.

Хэдийгээр эдгээр хөрвүүлэлтүүд их хэмжээгээр авч болно

маш их цаг хугацаа, хүчин чармайлт гдүрэмдаг ч ихэнхдээ тэд хэзээ ч зөв ажилладаггүй.

Хоёрдугаарт, та дунд програмын бүтээгдэхүүнийг ашиглан өөрийн хэрэглээний код дахь объектуудыг харилцааны загварт саадгүй буулгаж болно.

Харилцааны мэдээллийн сан маш их тархсан тохиолдолд энэ нь илүү сайн шийдэл юм.

ОО өгөгдлийн сан нь объектын тогтвортой байдлын хувьд харилцааны мэдээллийн сангаас хамаагүй илүү үр дүнтэй байдаг гэсэн мдүрэман байж магадгүй юм.

Үнэн хэрэгтээ олон хөгжүүлэлтийн системүүд энэ үйлчилгээг саадгүй хангадаг.

Объектын тогтвортой байдал- Object Persistence
Объектын тогтвортой байдал гэдэг нь тухайн объектын төлөв байдлыг хадгалах, дараа нь сэргээх, ашиглах боломжтой гэсэн ойлголтыг хэлдэг.

Тогтворгүй объект нь хамрах хүрээнээс гарах үед үндсэндээ үхдэг. Жишээлбэл, объектын төлөвийг мэдээллийн санд хадгалах боломжтой.

Гэсэн хэдий ч өнөөгийн бизнесийн орчинд объект хоорондын хамаарлын зураглал нь маш сайн шийдэл юм.

Олон компаниуд эдгээр технологийг нэгтгэсэн. Компани нь үндсэн фрэймийн өгөгдөл бүхий вэбсайтын интерфейстэй байх нь түгээмэл байдаг.

Хэрэв та бүхэлдээ ОО систем үүсгэвэл ОО мэдээллийн сан нь ашигтай (мөн илүү сайн гүйцэтгэлтэй) сонголт байж болох юм;

Гэсэн хэдий ч ОО өгөгдлийн сангууд ОО хэлнүүдийн өсөлттэй ойролцоо хаана ч байгаагүй.

Бие даасан програм
Шинэ ОО програмыг эхнээс нь үүсгэх үед ч хуучин өгөгдлөөс зайлсхийх нь тийм ч хялбар биш байж магадгүй юм.

Энэ нь шинээр үүсгэгдсэн ОО программ ч гэсэн бие даасан програм биш байх магадлалтай тул харилцааны мэдээллийн санд (эсвэл бусад өгөгдөл хадгалах төхөөрөмж) хадгалагдсан мэдээллийг солилцох шаардлагатай болдогтой холбоотой юм.

Өгөгдлийн сангийн жишээ рүү буцъя.

Зураг 2.2-т классд нийтийн интерфейсийг харуулсан бөгөөд өөр юу ч байхгүй.

Мэдээжийн хэрэг, энэ класс дуусмагц илүү олон дүрмүүдыг агуулсан байх болно, мөн шинж чанаруудыг агуулсан байх болно.

Гэсэн хэдий ч, энэ классйг ашиглаж байгаа програмист хүний хувьд эдгээр хувийн дүрэм, шинж чанаруудын талаар юу ч мэдэх шаардлагагүй.

Та олон нийтийн дүрмүүдын дотор код ямар харагддагийг мэдэх шаардлагагүй.

Та зүгээр л интерфэйсүүдтэй хэрхэн харилцахаа мэдэх хэрэгтэй.

Энэхүү нийтийн интерфэйсийн код ямар байх вэ (бид Oracle мэдээллийн сангийн жишээнээс эхэлнэ гэж бодъё)?

Open() дүрмийг харцгаая:

```
public void open(String Name){

        /* Some application-specific processing */

        /* call the Oracle API to open the database */

        /* Some more application-specific processing */

};
```

Энэ тохиолдолд та програмистын малгай өмссөн тул нээлттэй дүрэм нь String-ийг параметр болгон шаарддаг гэдгийг ойлгосон.

Өгөгдлийн сангийн файлыг төлөөлөх нэрийг дамжуулдаг боловч энэ жишээн дээр Нэр нь тодорхой мэдээллийн санд хэрхэн дүрслэгдсэнийг тайлбарлах нь чухал биш юм.

Энэ бол бидний мэдэх ёстой зүйл юм. Одоо интерфэйсийг үнэхээр гайхалтай болгодог хөгжилтэй зүйлс ирлээ!

Хэрэглэгчдээ бухимдуулахын тулд мэдээллийн сангийн хэрэгжүүлэлтийг өөрчилье. Өнгөрсөн шөнө бид бүх өгөгдлийг Oracle мэдээллийн сангаас SQLAnywhere мэдээллийн сан руу хөрвүүлсэн (бид цочмог болон архаг өвдөлтийг тэвчсэн).

Бид хэдэн цаг зарцуулсан ч бид үүнийг хийсэн.

Одоо код дараах байдалтай байна.

```
public void open(String Name){

        /* Some application-specific processing

        /* call the SQLAnywhere API to open the database */

        /* Some more application-specific processing */

};
```

Өнөө өглөө нэг ч хэрэглэгч гомдоллосонгүй.

Учир нь хэрэгжүүлэлт өөрчлөгдсөн ч интерфейс өөрчлөгдөөгүй! Хэрэглэгчийн хувьд дуудлага ижил хэвээр байна.

Хэрэгжүүлэхийн тулд кодын өөрчлөлт хийхэд багагүй ажил шаардагдаж магадгүй (мөн нэг мөрийн кодын өөрчлөлттэй модулийг дахин бүтээх шаардлагатай болно),

гэхдээ энэ DataBaseReader классыг ашигладаг програмын кодын нэг мөрийг өөрчлөх шаардлагагүй.

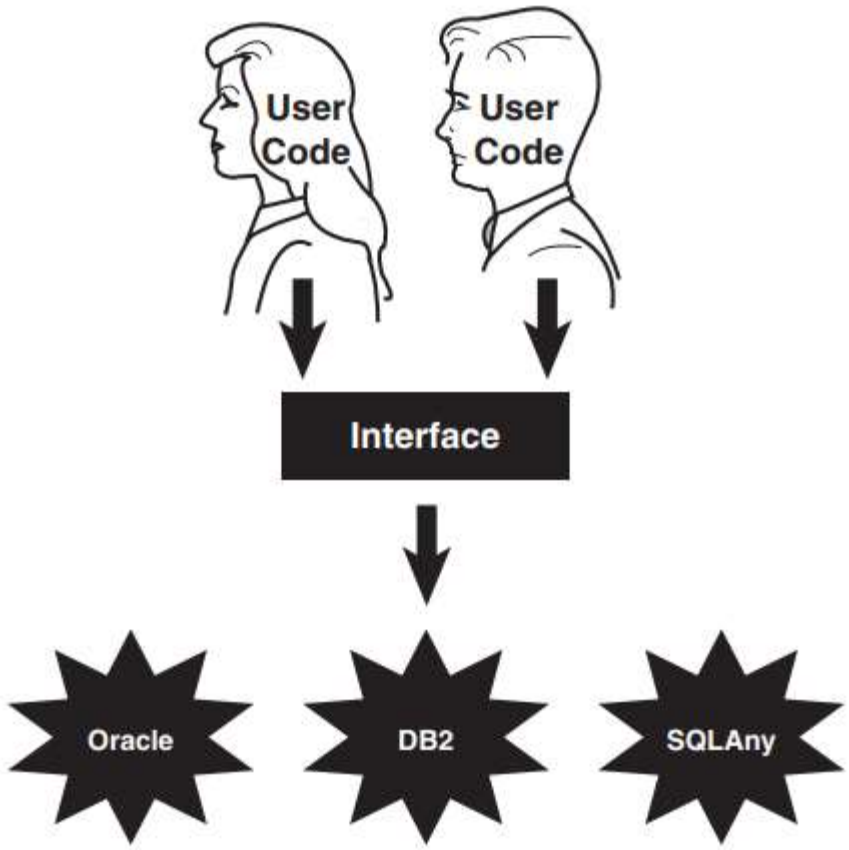
Кодын дахин эмхэтгэл
Динамикаар ачаалагдсан классуд нь ажиллах үед ачаалагддаг бөгөөд гүйцэтгэх боломжтой файлд статик байдлаар холбогддоггүй.

Java болон .NET зэрэг динамикаар ачаалагдсан классудыг ашиглах үед хэрэглэгчийн классудыг дахин хөрвүүлэх шаардлагагүй болно.

Гэсэн хэдий ч C++ гэх мэт статик холбоотой хэлнүүдэд шинэ класс оруулахын тулд холбоос шаардлагатай.

Хэрэглэгчийн интерфэйсийг хэрэгжүүлэлтээс салгаснаар бид зам дээр маш их толгойн өвчинг аварч чадна.

Зураг 2.3-т мэдээллийн сангийн хэрэгжүүлэлт нь зөвхөн интерфейсийг хардаг эцсийн хэрэглэгчдэд ил тод байна.



Зураг 2.3 Интерфайс

2. Интерфейс зохиохдоо хийсвэр сэтгэлгээг ашиглах
ОО програмчлалын гол давуу талуудын нэг нь классудыг дахин ашиглах боломжтой байдаг.

Ерөнхийдөө дахин ашиглах боломжтой классуд нь бетоноос илүү хийсвэр интерфейстэй байдаг.

Бетон интерфейс нь маш тодорхой байх хандлагатай байдаг бол хийсвэр интерфейс нь илүү ерөнхий байдаг. Гэсэн хэдий ч өндөр хийсвэр интерфейс нь өндөр нарийвчлалтай интерфэйсээс илүү ашигтай гэдгийг хэлэх нь ихэвчлэн үнэн байдаг ч үргэлж тийм байдаггүй.

Дахин ашиглах боломжгүй маш хэрэгтэй, бетонон класс бичих боломжтой.

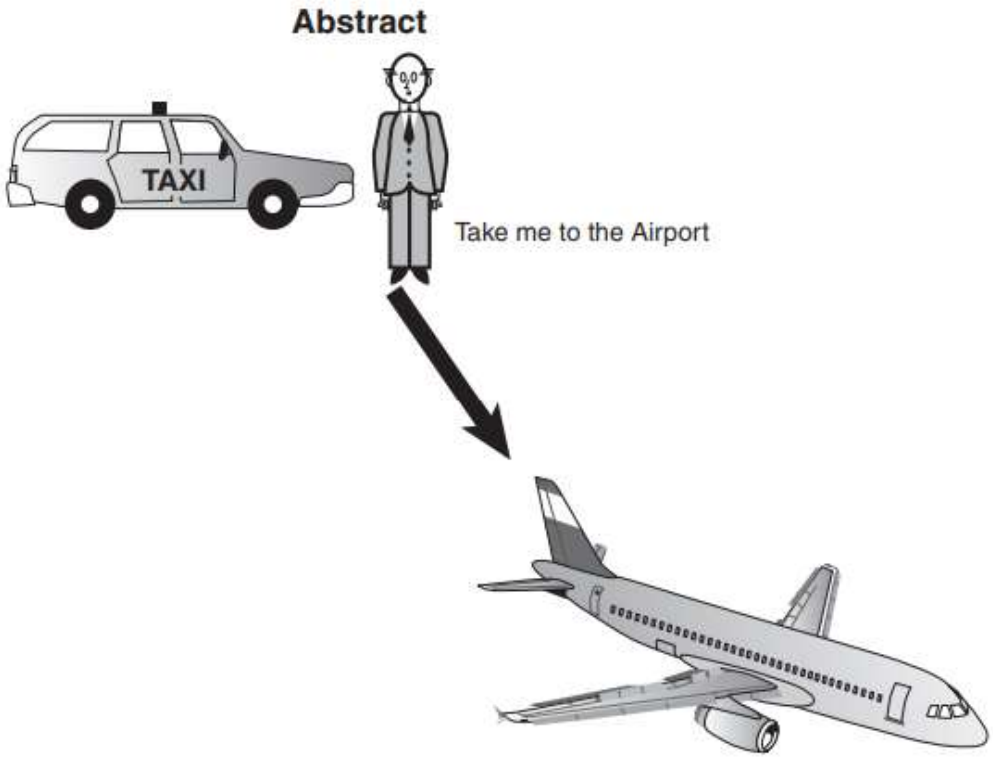
Энэ нь байнга тохиолддог бөгөөд зарим тохиолдолд буруу зүйл байхгүй.

Гэсэн хэдий ч бид одоо дизайны бизнест ажиллаж байгаа бөгөөд ОО-ийн бидэнд санал болгож буй давуу талыг ашиглахыг хүсч байна.

Тиймээс бидний зорилго бол хийсвэр, дахин ашиглах боломжтой классудыг зохион бүтээх бөгөөд үүний тулд бид өндөр хийсвэр хэрэглэгчийн интерфейсийг зохион бүтээх болно.

Хийсвэр болон тодорхой интерфейсийн ялгааг харуулахын тулд таксины объектыг бүтээцгээе.

Зурагт үзүүлсэн шиг "баруун эргэх", "зүүн эргэх", "эхлэх", "зогсоох" гэх мэт тусдаа интерфэйстэй байснаас "намайг нисэх онгоцны буудал руу хүргэж өг" гэх мэт интерфейстэй байх нь илүү ашигтай байдаг. Зураг 2.4, хэрэглэгчийн хийхийг хүссэн зүйл бол онгоцны буудал руу очих явдал юм.



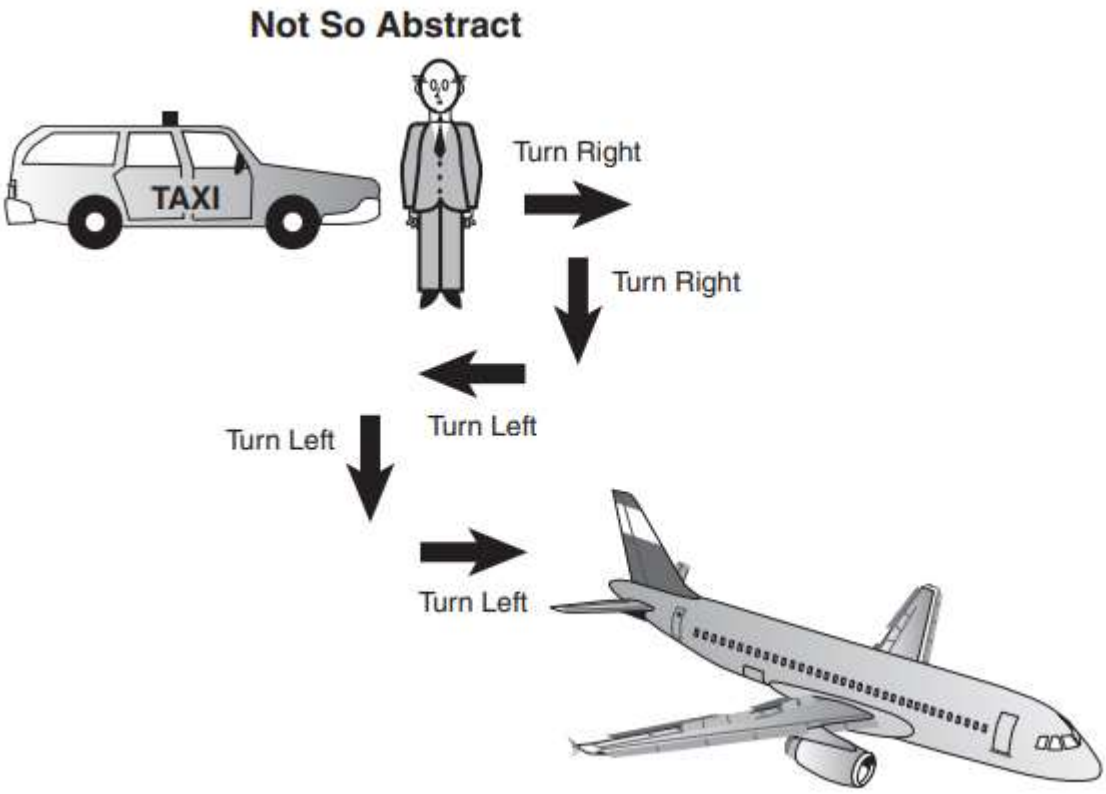
Зураг 2.4 Abstract interface

Зочид буудлаасаа гарч ирээд таксины арын суудал руу цүнхээ шидээд суухад таксичин чам руу эргэж хараад:

"Чи хаашаа явмаар байна?" гэж хариулна уу, "Гуйя

намайг онгоцны буудал руу авааач." (Мэдээжийн хэрэг, энэ хотод зөвхөн нэг томоохон нисэх онгоцны буудал байдаг гэж үздэг.

Чикагод та "Намайг Мидуэйн нисэх онгоцны буудал руу авааач" эсвэл "Намайг O'Харе руу авааач" гэж хэлэх хэрэгтэй.) Та өөрөө нисэх онгоцны буудал руу хэрхэн хүрэхээ мэдэхгүй байж магадгүй, тэгсэн ч гэсэн та үүнийг мэдэхгүй байж магадгүй юм. t Зураг 2.5-т үзүүлсэн шиг таксины жолоочид хэзээ эргэх, аль чиглэлд эргэхээ хэлэхийг хүсч байна.



Зураг 2.5 Not so abstract interface

Такси жинхэнэ жолоодлогыг хэрхэн хэрэгжүүлэх нь зорчигч танд хамаагүй. (Мэдээж хэрэг, таксичин хуурч таныг нисэх онгоцны буудал хүртэл хол замд хүргэж өгвөл үнэ нь асуудал болж магадгүй юм.)

Одоо хийсвэр болон дахин ашиглах хоёрын хоорондын холбоо хаанаас орж ирдэг вэ?

Эдгээр хоёр хувилбарын аль нь хийсвэр эсвэл тийм ч хийсвэр биш гэдгийг дахин ашиглах боломжтой гэж өөрөөсөө асуугаарай.

Илүү энгийнээр хэлбэл, "Намайг нисэх онгоцны буудал руу авааач" эсвэл "Баруун, дараа нь баруун, дараа нь зүүн, дараа нь зүүн, дараа нь зүүн" гэсэн хэллэгийг дахин ашиглах боломжтой юу? Мэдээжийн хэрэг, эхний хэллэгийг дахин ашиглах боломжтой.

Та таксинд суугаад нисэх онгоцны буудал руу явахыг хүссэн үедээ аль ч хотод ашиглах боломжтой.

Хоёрдахь хэллэг нь зөвхөн тодорхой тохиолдолд ажиллах болно.

Тиймээс, "Намайг нисэх онгоцны буудал руу авааач" гэсэн хийсвэр интерфейс нь ерөнхийдөө Чикаго, Нью-Йорк, Кливленд зэрэгт хэрэгжих нь өөр байж болох сайн, дахин ашиглах боломжтой ОО дизайн хийх дүрэм зам юм.

3. Хэрэглэгчдэд боломжит хамгийн бага интерфэйсийг өгөх (Giving the user the minimal interface possible)

Классудыг зохион бүтээхдээ үндсэн дүрэм нь хэрэглэгчийг классын дотоод үйл ажиллагааны талаар аль болох бага мэдлэгээр хангах явдал юм.

Үүнийг хийхийн тулд дараах энгийн дүрмийг дагаж мөрдөөрэй.

- Хэрэглэгчдэд зөвхөн хэрэгцээтэй зүйлээ өг.

Үнэн хэрэгтээ энэ нь класс аль болох цөөн интерфейстэй гэсэн үг юм.

Класс зохион бүтээж эхлэхдээ хамгийн бага интерфэйсээс эхэл.

Классын загвар нь давталттай байдаг тул та интерфэйсийн хамгийн бага багц хангалтгүй байж болохыг удахгүй олж мэдэх болно.

Энэ зүгээр. Хэрэглэгчдэд хэрэгцээнээс илүү интерфэйс өгөхөөс илүү хэрэглэгчдэд үнэхээр хэрэгтэй байгаа тул интерфэйс нэмэх нь дээр.

Хэрэглэгч тодорхой интерфэйстэй байх нь асуудалтай байх тохиолдол байдаг. Жишээлбэл, та зөвхөн мэдэх шаардлагатай бүх хэрэглэгчдэд цалингийн мэдээлэл өгөх интерфейсийг хүсэхгүй байна.

Одоогийн байдлаар програм хангамжийн жишээг харуулахын тулд техник хангамжийн жишээг ашиглая.

Хэрэглэгчид дэлгэц, гаргүй компьютерийн хайрцаг өгнө гэж төсөөлөөд үз дээ.

Мэдээжийн хэрэг, компьютер бага зэрэг ашиг тустай байх болно. Та сая хэрэглэгчдэд PC-ийн интерфейсийн хамгийн бага багцыг өгсөн.

Мэдээжийн хэрэг, энэ хамгийн бага багц нь хангалтгүй бөгөөд тэр даруй

интерфэйсүүдийг нэмэх шаардлагатай болдог.

- Нийтийн интерфэйс нь хэрэглэгчид юу хандаж болохыг тодорхойлдог.

Хэрэв та эхлээд интерфэйсүүдийг хувийн болгосноор бүх классйг хэрэглэгчээс нуусан бол программистууд класс ашиглаж эхлэх үед та тодорхой дүрмүүдыг олон нийтэд нээлттэй болгохоос өөр дүрэмгүй болно-эдгээр дүрмүүд нь нийтийн интерфейс болно.

- Классудыг мэдээллийн системийн үүднээс бус хэрэглэгчийн үүднээс авч үзэх нь чухал.

Ихэнх тохиолдолд классын дизайнерууд (өөр төрлийн програм хангамжийг дурдахгүй) класса тодорхой технологийн загварт нийцүүлэхийн тулд загвар зохион бүтээдэг.

Загвар зохион бүтээгч нь хэрэглэгчийн байр сууринаас хандсан ч гэсэн энэ нь техникч хэрэглэгчийн үзэл бодол хэвээр байх бөгөөд класс нь үүнийг технологийн дагуу ажиллуулахад чиглэгдсэн байдаг.

Хэрэглэгчийн хувьд ашиглахад хялбар байдлаас биш.

- Класс зохиохдоо зөвхөн хөгжүүлэгчид гэлтгүй үүнийг ашиглах хүмүүстэй хамт тавигдах шаардлага, дизайныг сайтар нягталж үзээрэй.

Системийн прототипийг бүтээх үед класс нь хувьсан өөрчлөгдөж, шинэчлэх шаардлагатай болно.

Хэрэглэгчдийг тодорхойлох

Таксины жишээг дахин харцгаая. Хэрэглэгчид нь уг системийг ашиглах хүмүүс гэдгийг бид аль хэдийн шийдсэн.

Энэ нь ойлгомжтой асуулт бол хэрэглэгчид хэн бэ?

Эхний түлхэц бол үйлчлүүлэгчид хэлэх явдал юм.

Энэ нь зөвхөн тал орчим нь зөв юм.

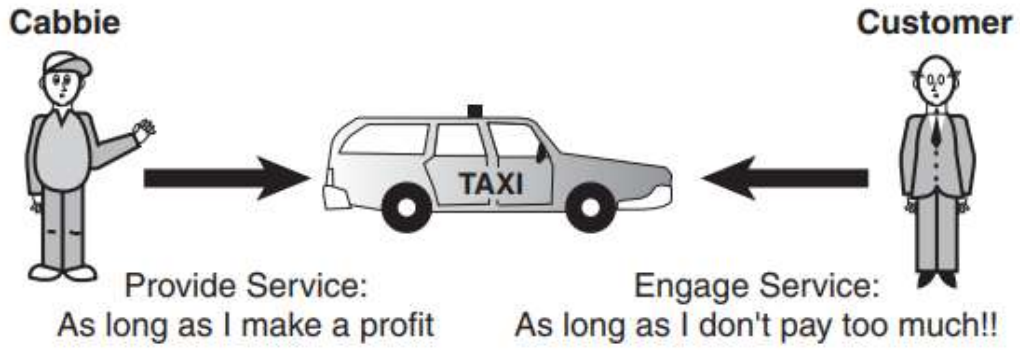
Хэдийгээр үйлчлүүлэгчид нь мэдээжийн хэрэг хэрэглэгчид боловч такси нь үйлчлүүлэгчдээ амжилттай үйлчилгээ үзүүлэх чадвартай байх ёстой. Өөрөөр хэлбэл, "Намайг нисэх онгоцны буудалд үнэ төлбөргүй хүргэж өгөөч" гэх мэт үйлчлүүлэгчдэд таалагдах интерфэйсийг өгөх нь таксины үйлчилгээнд сайнаар нөлөөлөхгүй.

Тиймээс бодит байдал дээр бодитой, ашиглах боломжтой интерфэйсийг бий болгохын тулд үйлчлүүлэгч болон таксины аль алиныг нь хэрэглэгч гэж үзэх ёстой.

Програм хангамжийн аналогийн хувьд хэрэглэгчид программистыг тодорхой функцээр хангахыг хүсч магадгүй гэдгийг анхаарч үзээрэй.

Гэсэн хэдий ч, хэрэв програмист хүсэлтийг техникийн хувьд боломжгүй гэж үзвэл програмист хичнээн туслахыг хүссэн ч хүсэлтийг хангаж чадахгүй.

Товчхондоо, таксины объект руу мессеж илгээдэг аливаа объектыг хэрэглэгч гэж үздэг (тиймээ, хэрэглэгчид ч гэсэн объект юм). Зураг 2.6-д таксичин хэрхэн үйлчилгээ үзүүлж байгааг харуулав.



Зураг 2.6 Үйлчилгээ үзүүлэх

Лавшруулж харах

Такси нь бас объект байх магадлалтай.

Обьектийн зан байдал-Object Behavior

Хэрэглэгчдийг тодорхойлох нь дасгалын зөвхөн нэг хэсэг юм.

Хэрэглэгчдийг тодорхойлсоны дараа та объектуудын зан төлөвийг тодорхойлох ёстой.

Бүх хэрэглэгчдийн байр сууринаас объект бүрийн зорилго, зөв ажиллахын тулд юу хийх ёстойг тодорхойлж эхэл.

Анхны сонголтуудын ихэнх нь нийтийн интерфэйсийн эцсийн зүсэлтээс гарахгүй гэдгийг анхаарна уу.

Эдгээр сонголтыг UML UseCases гэх мэт янз бүрийн дүрмүүдыг ашиглан шаардлагуудыг цуглуулах замаар тодорхойлдог.

Орчны хязгаарлалт-Environmental Constraints

Гилберт, Маккарти нар Java хэл дээрх Объект хандалтат дизайн номондоо хүрээлэн буй орчин нь объектын хийж чадах зүйлд хязгаарлалт тавьдаг гэж онцлон тэмдэглэжээ.

Үнэн хэрэгтээ байгаль орчны хязгаарлалт нь бараг үргэлж хүчин зүйл болдог. Компьютерийн техник хангамж нь програм хангамжийн ажиллагааг хязгаарлаж болзошгүй.

Жишээлбэл, систем нь сүлжээнд холбогдоогүй эсвэл компани тодорхой төрлийн принтер ашиглаж болно.

Таксины жишээн дээр, онгоцны буудал руу хурдан хүрэх замыг өгсөн ч гүүр гарсан тохиолдолд кабин нь зам дээр явж чадахгүй.

Олон нийтийн интерфейсийг тодорхойлох

Хэрэглэгчид, объектын зан төлөв, хүрээлэн буй орчны талаар цуглуулсан бүх мэдээллийг ашиглан та хэрэглэгчийн объект бүрийн нийтийн интерфейсийг тодорхойлох хэрэгтэй.

Тиймээс та таксины объектыг хэрхэн ашиглах талаар бодож үзээрэй.

-Таксинд суу.

-Хаашаа явахыг таксины хүнд хэлээрэй.

- Таксинд мөнгө төл.

-Таксинд цайны мөнгө өг.

-Таксинаас буу.

Таксины объектыг ашиглахын тулд юу хийх хэрэгтэй вэ?

-Очих газартай.

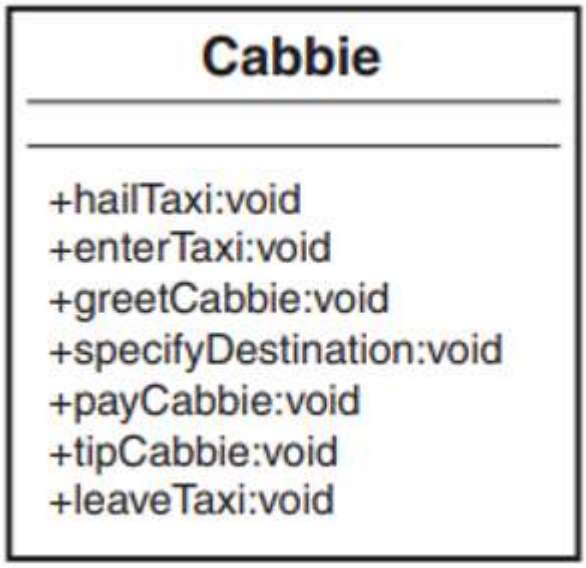
-Такси барина уу.

-Таксины мөнгөө төл.

Эхлээд та объектыг хэрхэн яаж барьж байгаа талаар биш харин хэрхэн ашиглаж байгаа талаар боддог.

Та уг объектод "Ачаа тээшээ тээшинд хийх" эсвэл "Таксины жолоочтой ухаангүй яриа өрнүүлэх" гэх мэт илүү олон интерфейс хэрэгтэй байгааг олж мэдэж болно.

” Зураг 2.7-д Cabbie классын боломжит дүрмүүдыг жагсаасан классын диаграммыг үзүүлэв.



Зураг 2.7-д Cabbie классын боломжит дүрмүүд

Үргэлж байдаг шиг, эцсийн интерфейсийг хадаж байна

давтагдах үйл явц.

Интерфейс бүрийн хувьд интерфэйс нь объектын үйл ажиллагаанд хувь нэмэр оруулж байгаа эсэхийг тодорхойлох ёстой.

Хэрэв тийм биш бол энэ нь шаардлагагүй байж магадгүй юм.

Олон ОО текстүүд интерфэйс бүрийг зөвхөн нэг зан төлөвийг загварчлахыг зөвлөж байна.

Энэ нь бид дизайныг хэр хийсвэр болгохыг хүсч байна вэ гэсэн асуулт руу буцаана.

Хэрэв бидэнд enterTaxi() гэсэн интерфэйс байгаа бол бид таксины төлбөрийг төлөхийн тулд enterTaxi()-г логиктой байлгахыг хүсэхгүй нь лавтай.

Хэрэв бид үүнийг хийвэл загвар нь зарим талаараа логикгүй төдийгүй, классын хэрэглэгч таксинд төлөхийн тулд юу хийх ёстойг хэлж чадахгүй.

Хэрэгжүүлэлтийг тодорхойлох
Нийтийн интерфейсийг сонгосны дараа та хэрэгжүүлэлтийг тодорхойлох хэрэгтэй.

Класса зохион бүтээж, классийг зөв ажиллуулахад шаардагдах бүх дүрмийг хэрэглэсний дараа тухайн классйг хэрхэн ажилд оруулах онцлогийг авч үздэг.

Техникийн хувьд нийтийн интерфейс биш бүх зүйлийг хэрэгжүүлэлт гэж үзэж болно.

Энэ нь хэрэглэгч аргын гарын үсэг зэрэг хэрэгжүүлэлтийн нэг хэсэг гэж тооцогддог дүрмүүдын аль нэгийг хэзээ ч харахгүй гэсэн үг юм

(энэ нь аргын нэр болон параметрийн жагсаалтыг агуулсан), мөн аргын доторх бодит код.

Классын дотооддоо ашигладаг хувийн дүрэмтай байж болно.

Аливаа хувийн дүрмийг хэрэглэгч хэзээ ч харахгүй тул түүнд хандах эрхгүй тул хэрэгжүүлэлтийн нэг хэсэг гэж үздэг. Жишээлбэл, класс нь changePassword() дүрэмтай байж болно; Гэсэн хэдий ч ижил класс нь нууц үгээ шифрлэдэг хувийн дүрэмтай байж болно.

Энэ дүрмийг хэрэглэгчээс нууж, зөвхөн changePassword() аргын дотроос дуудна.

Хэрэгжүүлэлт нь хэрэглэгчээс бүрэн далдлагдсан.

Нийтийн дүрмүүдын доторх код нь үнэндээ хэрэгжүүлэлтийн нэг хэсэг юм, учир нь хэрэглэгч үүнийг харж чадахгүй.

(Хэрэглэгч зөвхөн интерфэйсийн дуудлагын бүтцийг харах ёстой бөгөөд түүний доторх кодыг харахгүй.)

Энэ нь онолын хувьд хэрэгжсэн гэж үзэж буй бүх зүйл хэрэглэгчийн класстай хэрхэн холбогдоход нөлөөлөхгүйгээр өөрчлөгдөж болно гэсэн үг юм.

Энэ нь мэдээж хэрэг хэрэгжүүлэлт нь хэрэглэгчийн хүлээж буй хариултуудыг өгч байна гэж тооцдог.

Интерфэйс нь тухайн объектыг хэрэглэгч хэрхэн харж байгааг илэрхийлдэг бол хэрэгжүүлэлт нь уг объектын самар, боолт юм.

Хэрэгжүүлэлт нь тухайн объектын төлөв байдлыг илэрхийлэх кодыг агуулна.

Дүгнэлт

Энэ бүлэгт бид таныг ОО дүрэм замаар сэтгэх замд тань оруулах гурван чиглэлийг судалсан.

Үүнтэй холбоотой асуудлын тодорхой жагсаалт байхгүй гэдгийг санаарай

ОО бодлын үйл явц.

Юмыг ОО дүрэм замаар хийх нь шинжлэх ухаан гэхээсээ илүү урлаг юм. ОО сэтгэлгээг тодорхойлох өөрийн дүрэм замыг бодож үзээрэй.

3-р бүлэгт "Объект хандалтад чиглэсэн дэвшилтэт ойлголтууд"-д бид тухайн объект нь амьдралын мөчлөгтэй байдаг тухай өгүүлдэг: Энэ нь төрж, амьдардаг, үхдэг. Амьд байхдаа олон янзын төлөвт шилжиж болно.

Жишээлбэл, DataBaseReader объект нь нэг төлөвт байна

өгөгдлийн сан нь нээлттэй, хэрэв мэдээллийн сан хаалттай бол өөр төлөв байна. Үүнийг хэрхэн илэрхийлэх нь тухайн классын загвараас хамаарна.